

Time Series

Data Science

Time Series

- forecasting
- autocorrelation
- ARMA models
- time-series features
- temporal confounding
- deep learning for time series
- anomaly detection

Temporal Dependence

Time series observations are not independent.

Instead, present values depend on the past: $y_t \sim f(y_{t-1}, y_{t-2}, \dots)$

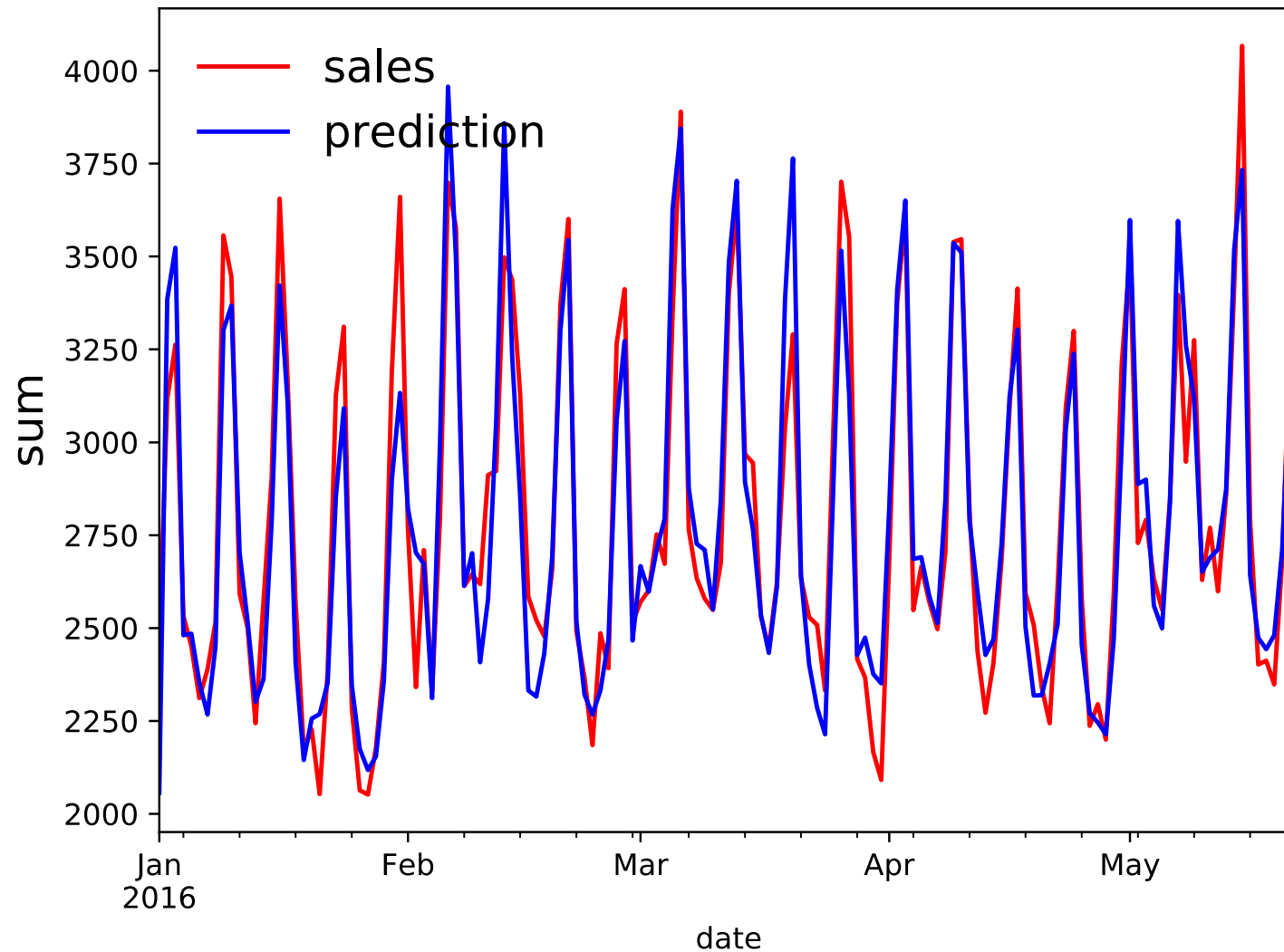
examples: weather, stock prices, sensor measurements, patient trajectories

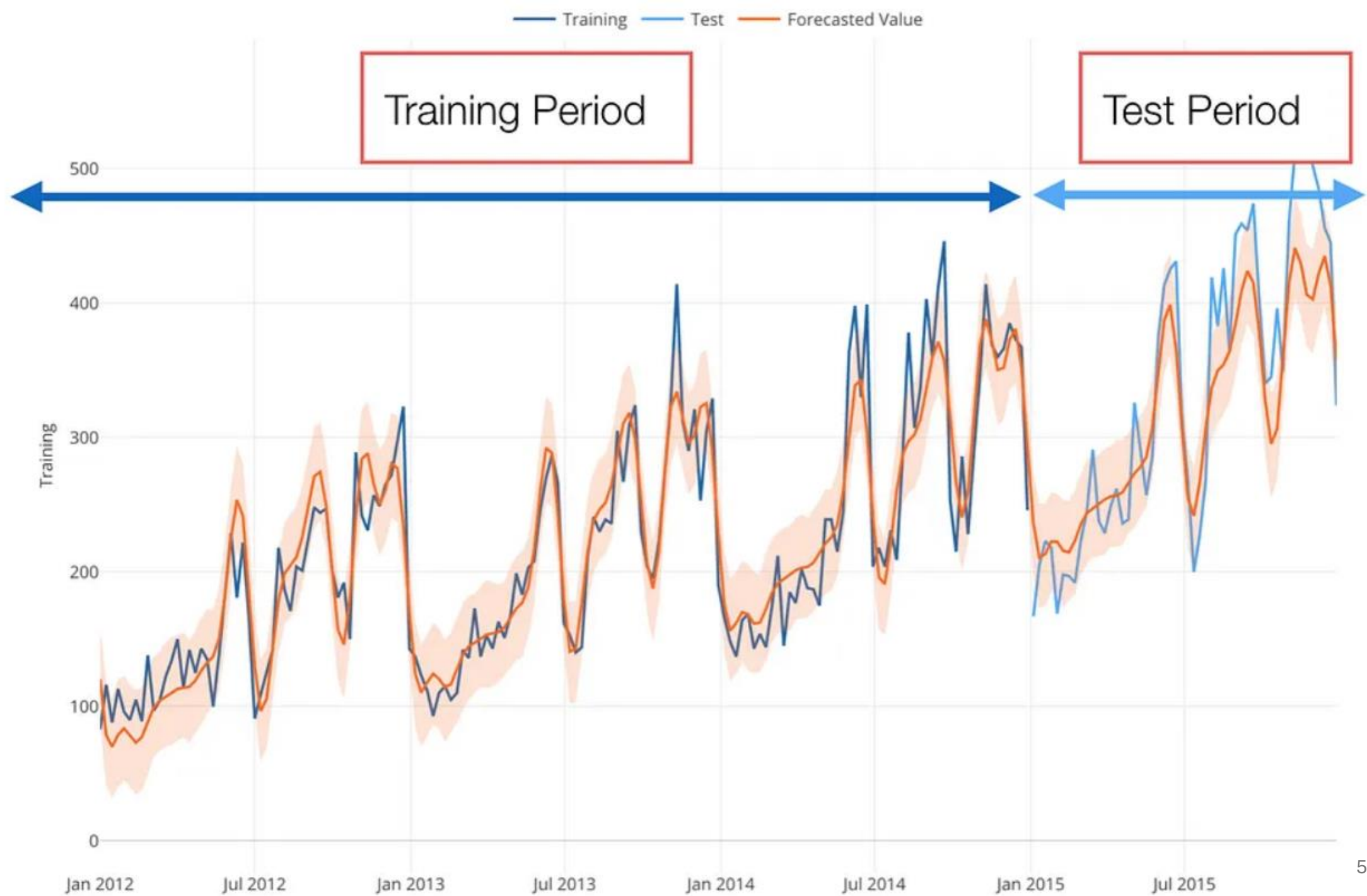
common temporal patterns:

- trend: long-term change
- seasonality: repeating cycles
- lag dependence: memory of past states

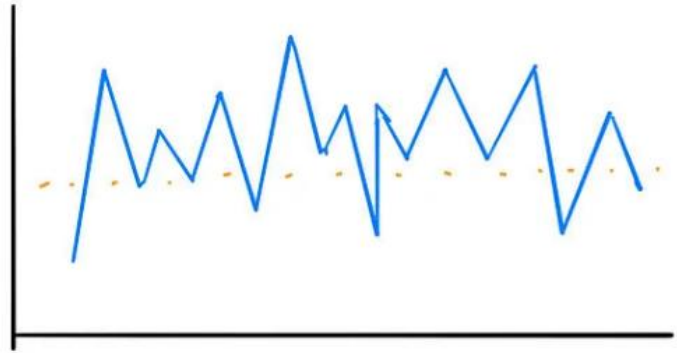
ignoring time structure → weaker models, misleading evaluation

Example: Forecasting



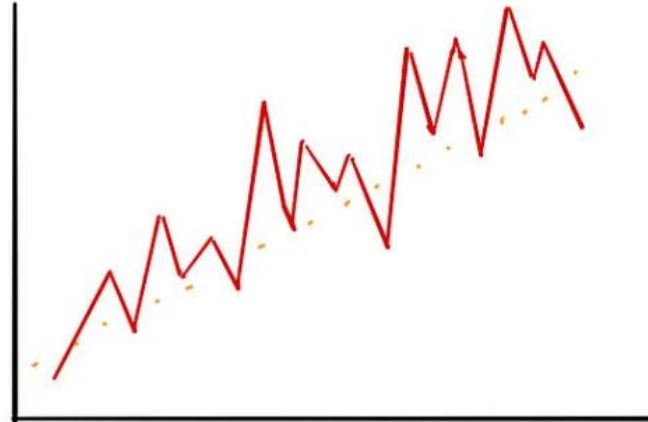


Statistical Patterns



STATIONARY

TREND



SEASONAL

Autocorrelation (ACF) and Partial Autocorrelation (PACF)

ACF: correlation with past values

$$\rho_k = \text{Corr}(y_t, y_{t-k})$$

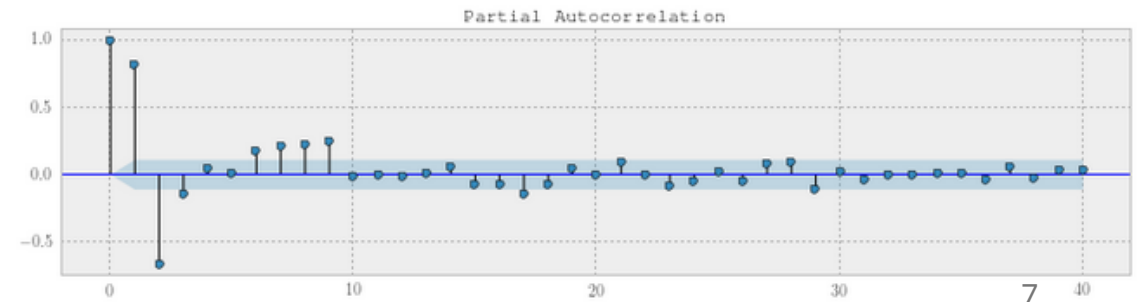
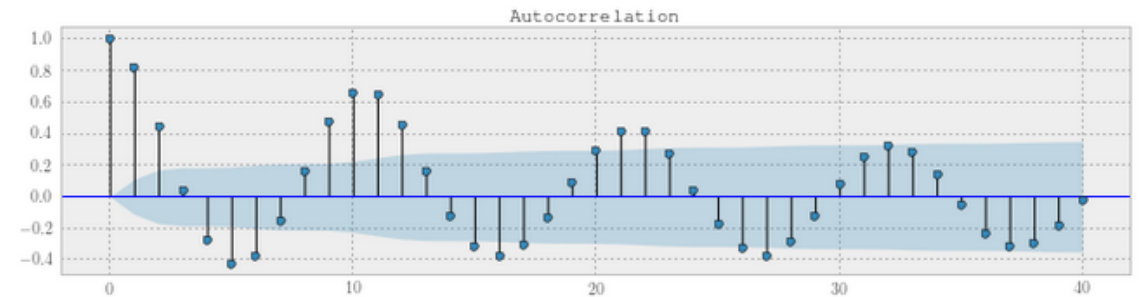
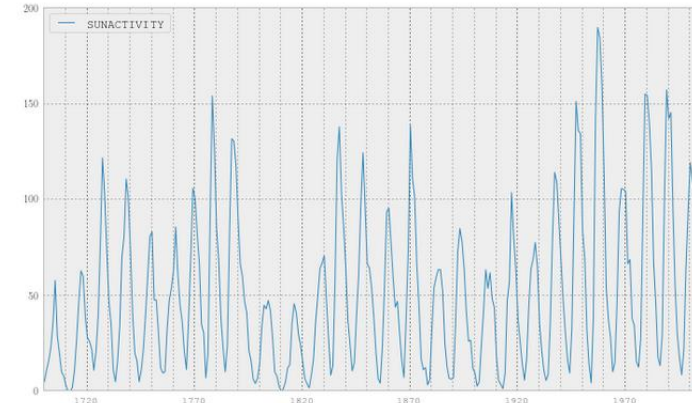
“How much does the past resemble the present?”

→ persistence, seasonality, moving-average structure

PACF: direct effect of lag k , controlling for shorter lags

$$\text{Corr}(y_t, y_{t-k} | y_{t-1}, \dots, y_{t-k+1})$$

→ autoregressive order, direct temporal dependencies



ARMA

AutoRegressive (AR) model:

predict from past observations (lagged values)

$$y_t = c + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \epsilon_t$$

Moving Average (MA) model:

predict from past errors (lagged shocks/residuals)

$$y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q}$$

$AR(p) + MA(q) \rightarrow ARMA(p,q)$ $\rightarrow$ decide on p and q by ACF/PCF

ARIMA

$$d = 1$$

$$d = 2: y'_t = y_t - 2 \cdot y_{t-1} + y_{t-2}$$

remove trend by differencing: $y'_t = y_t - y_{t-1}$

→ include integrated part: $AR(p) + I(d) + MA(q) \rightarrow ARMA(p, q)$

additional possible components:

SARIMA: ... seasonality

SARIMAX: ... exogenous

Types of Time Series Forecasting Information

auto-correlation:

- using past values of considered time series as function of lag
- identifying periodicity (seasonality) and trend

→ ARMA

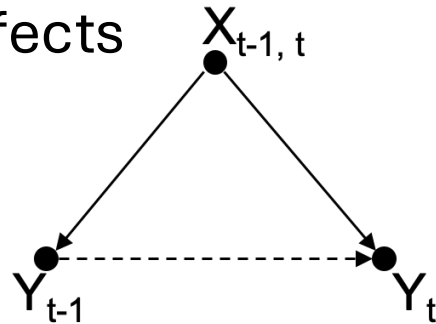
exogenous variables:

- using correlations of other variables to considered time series
- examples in demand forecasting: price, weather
- such effects lasting for some time create spurious auto-correlation

Temporal Confounding

auto-correlation often only spurious: common causes at consecutive times

→ no direct causal effects



specific lags such as day of week
can be interpreted both ways

advantages of learning direct causal effects from exogenous information instead of confounded auto-correlation:

- explainability
- long-term forecasting
- predictability of rare events

... ML ...

decomposition-based approaches:

- additive components: trend, seasonality, holidays, others (→ Generalized Additive Models)
- curve fitting (ML-like)
- example method: prophet
- natural way for multivariate model: training with several independent time series, distinguished by additional feature (e.g., product ID)

Feature Engineering for Time Series

Lag Features

create features from past values $x_{t-1}, x_{t-2}, x_{t-3}, \dots$

example: predict today's sales using sales yesterday, sales last week, ...

→ captures short- and long-term patterns

Rolling Statistics

compute aggregation statistics over a window (e.g., rolling mean, rolling sum)

example: average sales over last 7 days

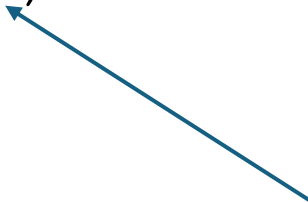
Exponential Smoothing

... exponentially weighted moving average:

$$\hat{y}_t = \alpha \cdot y_{t-1} + (1 - \alpha) \cdot \hat{y}_{t-1}$$

```
X.sort_values(['date'])  
.groupby(['store', 'item'])['y']  
.apply(lambda x: x.shift(3))  
.ewm(alpha=0.15).mean()
```

potentially further grouped
by important lag (like day of
week) or exogenous
variables (like promotion)



prediction horizon

Trend Features

capture long-term movement such as increasing/decreasing trend
difference between current and past values

Seasonality Features

capture repeating patterns

examples:

- day of week
- month
- holiday indicators

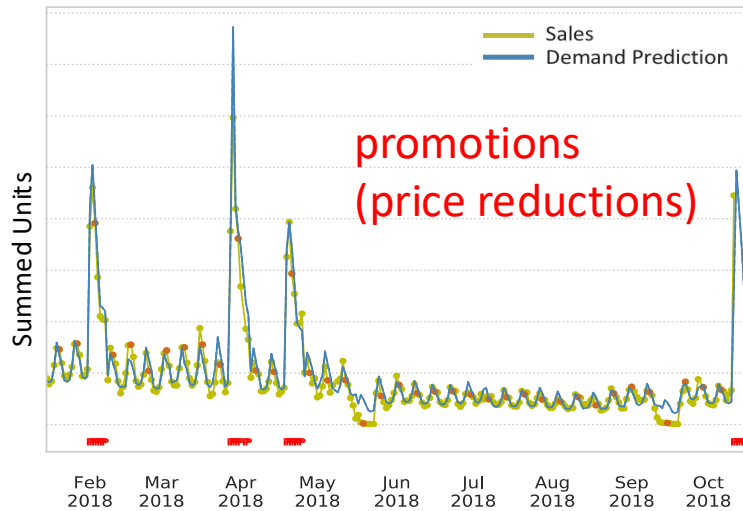
→ often critical in time series problems

Demand Forecasting with ML

many individual time series to consider

typical retail grocery chain:

- products (items): ~20k
- locations (stores): ~500
- daily/hourly aggregated sales



advantages of ML over traditional univariate time series forecasting:

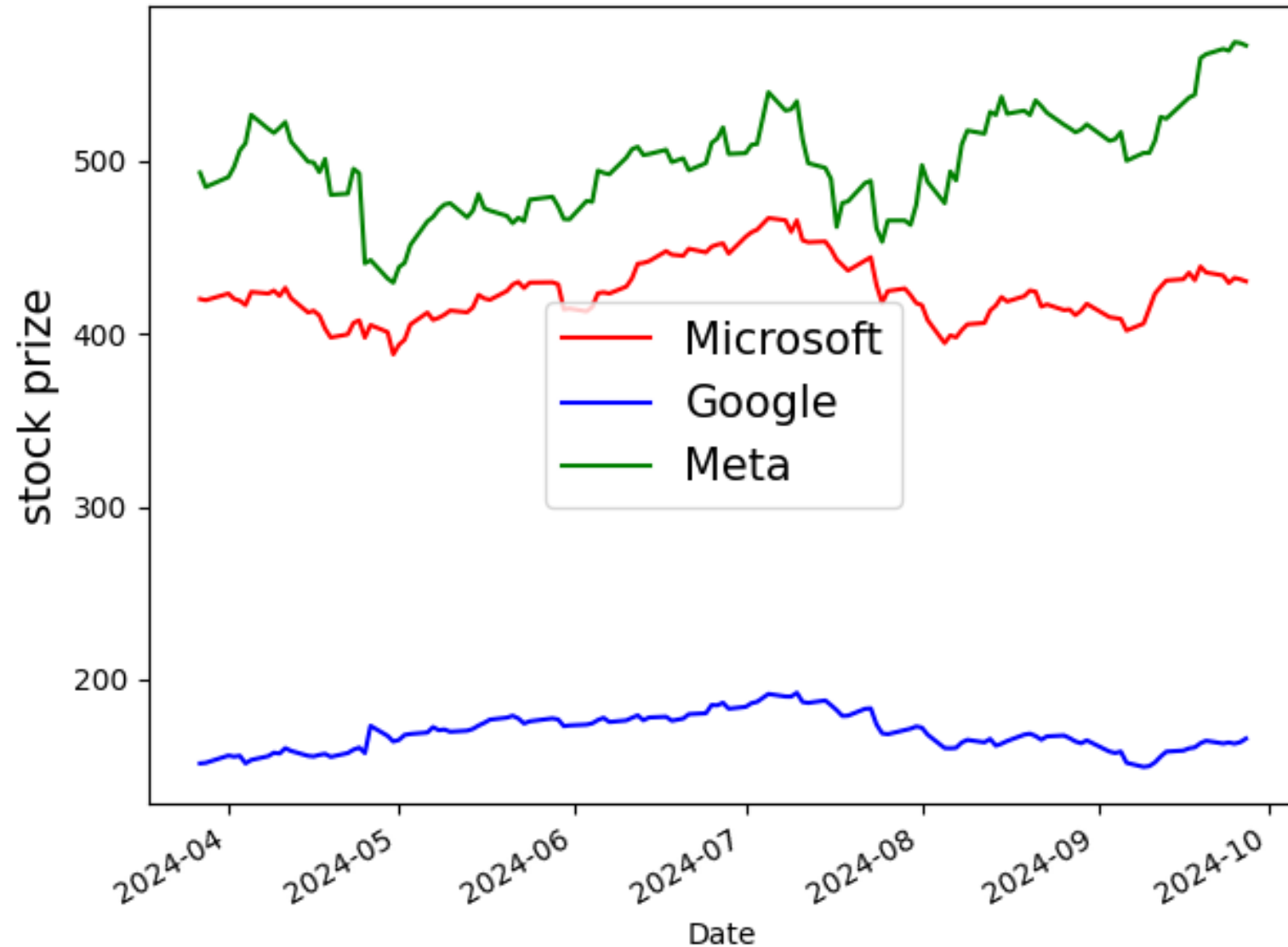
combined learning on all time series of product-location combinations (rather than separately optimizing individual time series)
→ **reduces variance** by exploiting commonalities

natural consideration of many exogenous variables (prices, promotions, holidays, weather, ...)
→ **reduces bias**

to be noted:

- categorical features important (products and locations → high cardinality)
- mainly multiplicative effects
- demand (approximately) following Poisson (or rather negative binomial) distribution

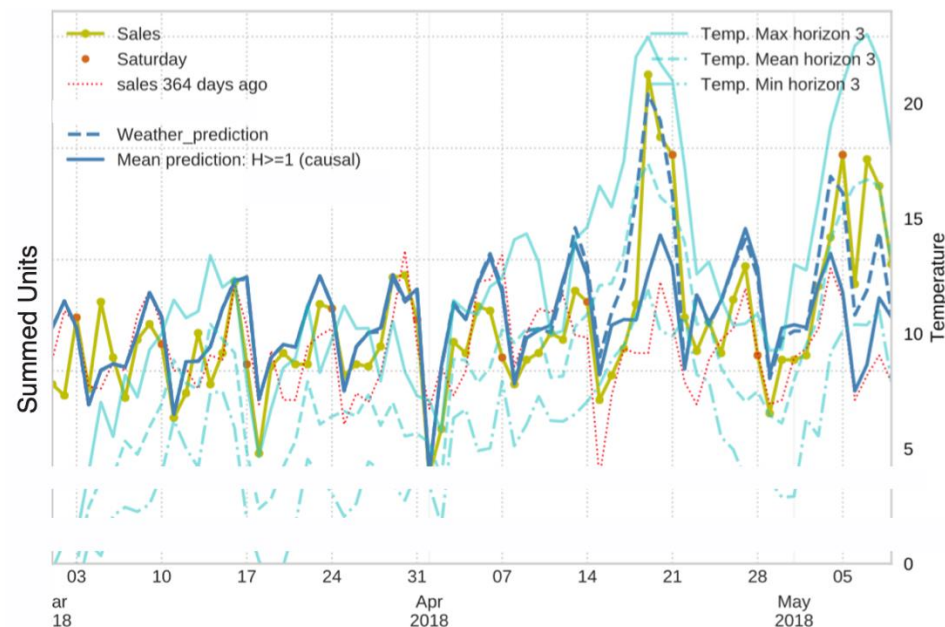
You can decrease uncertainty by looking at similar time series



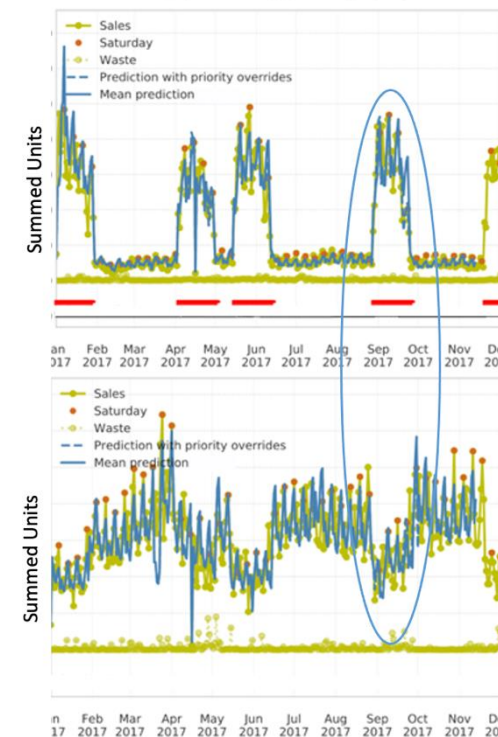
Example: Retail Features

Summer Weather

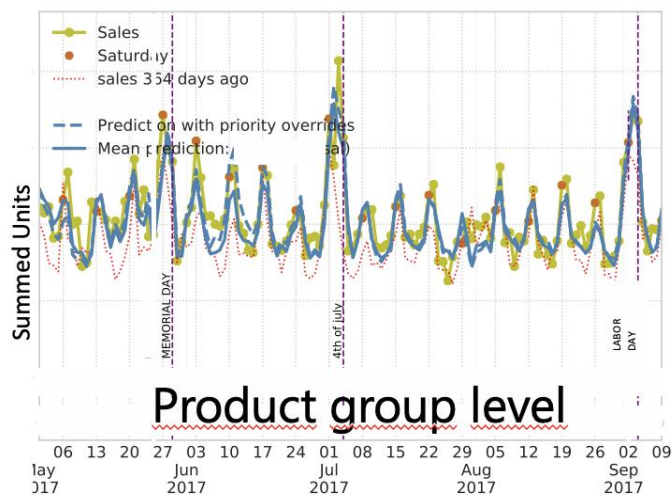
Sales
Demand Prediction



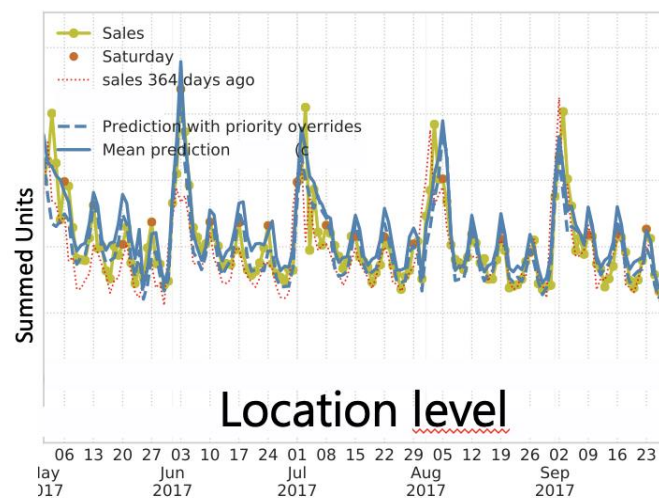
Cannibalization



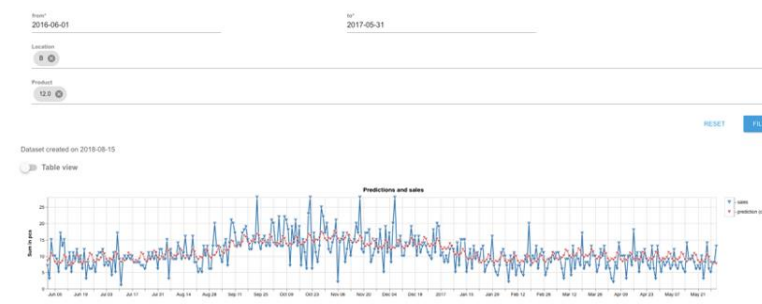
Events



Payday Effect



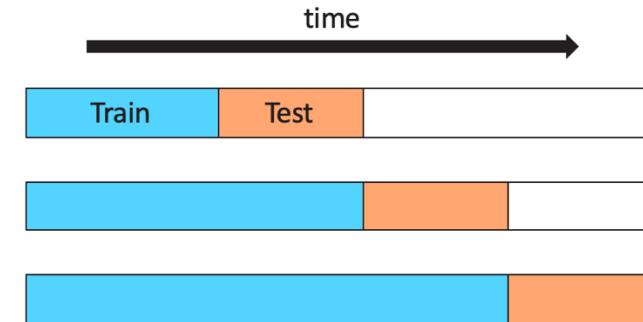
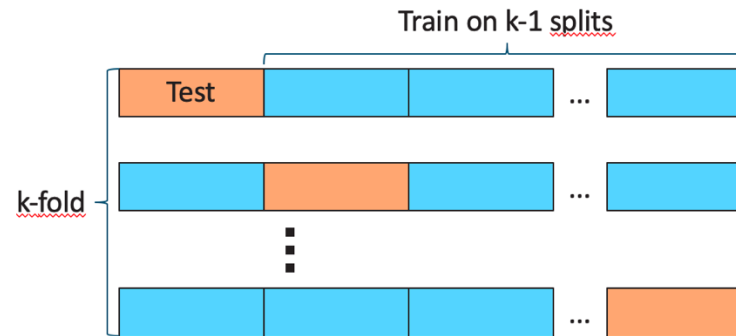
Seasonality



avoid time –based leakage: only use past data at prediction time

train-test splits in time:

cross-validation:



Beware of (Spurious) Auto-Correlation

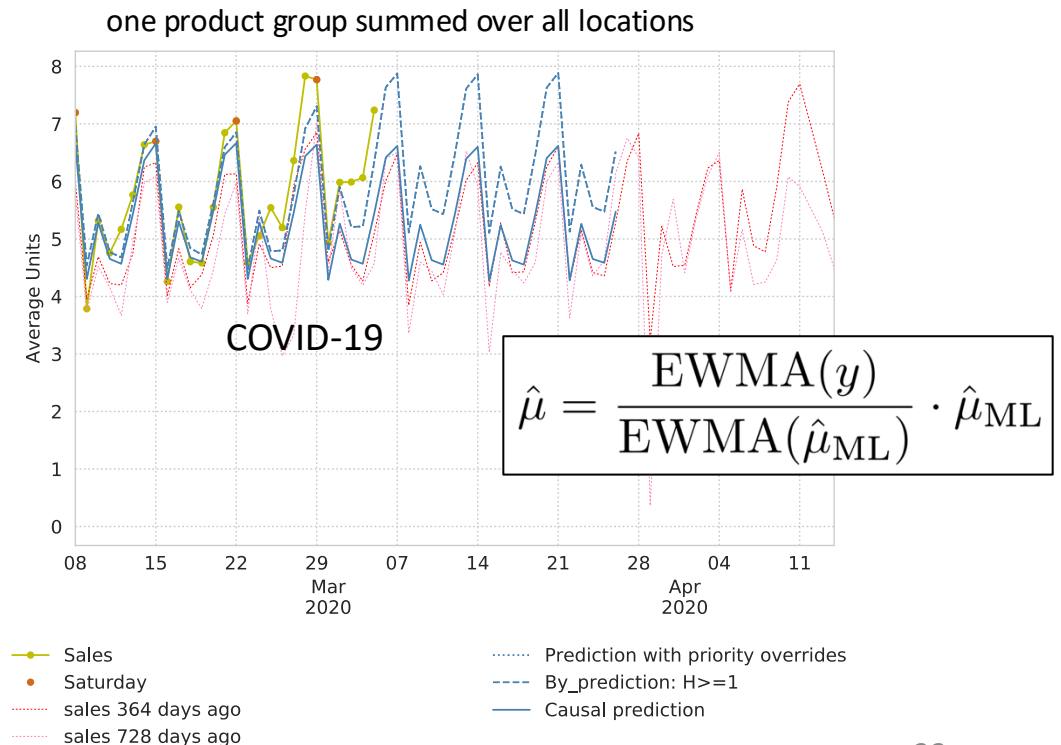
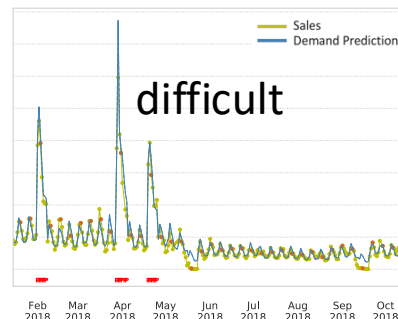
Most forecasting methods rely on lagged target information.

- univariate time series methods
- exponential smoothing features in ML model
- sequence modeling in deep learning (e.g., recurrent neural networks)

However, this makes learning of exogenous effects much harder.

- already blended in target auto-correlation
- delayed effects of short-lived structures/trends

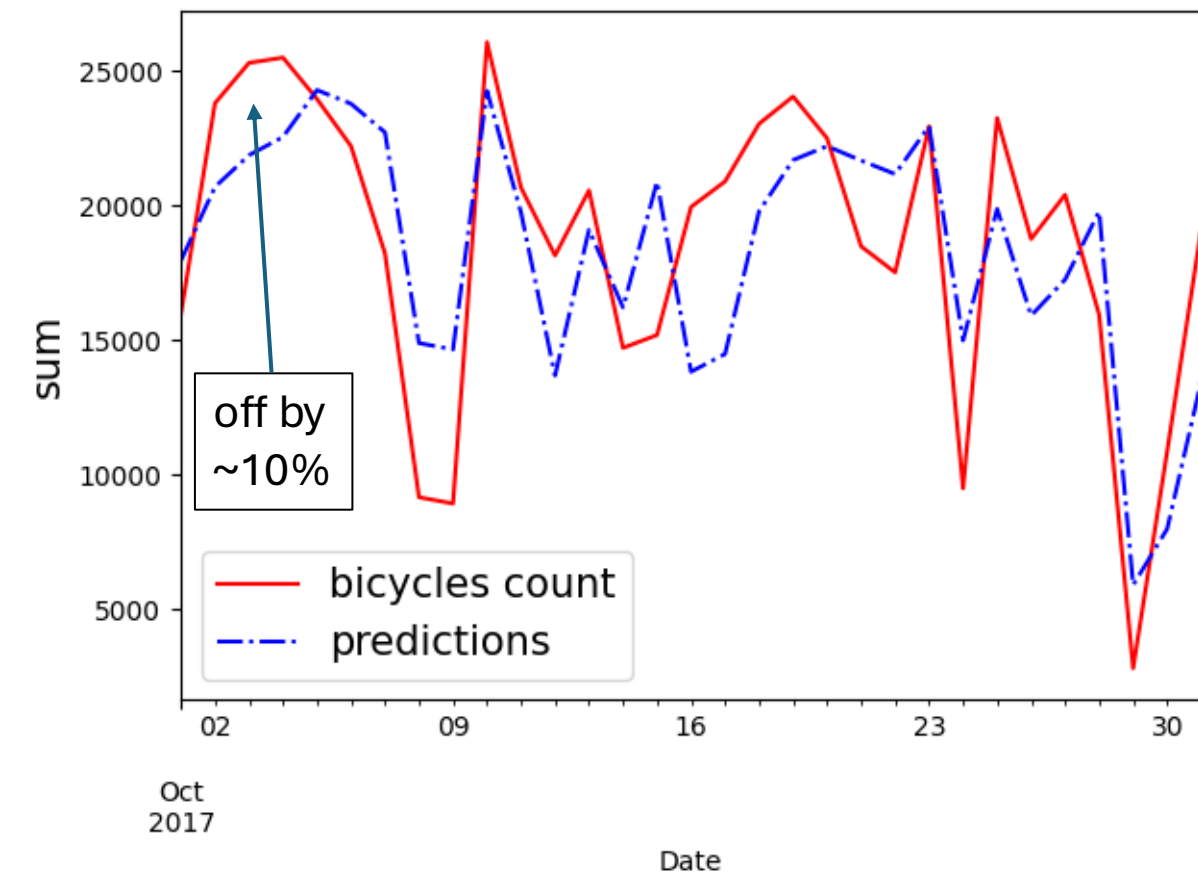
➤ simple solution: no use of target auto-correlation in ML model (but subsequent residual correction to capture missing trends)



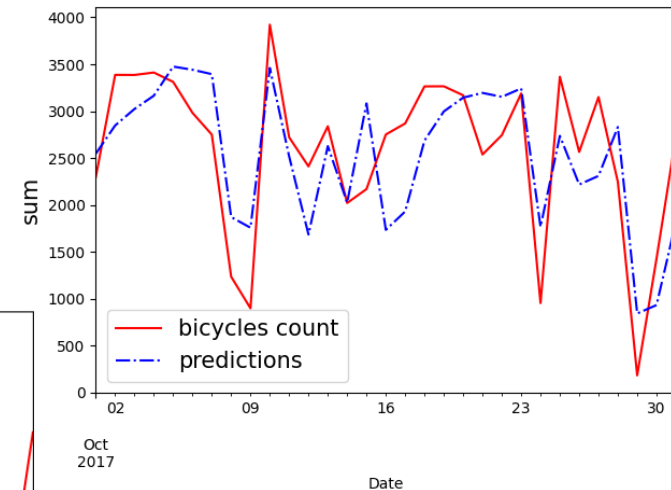
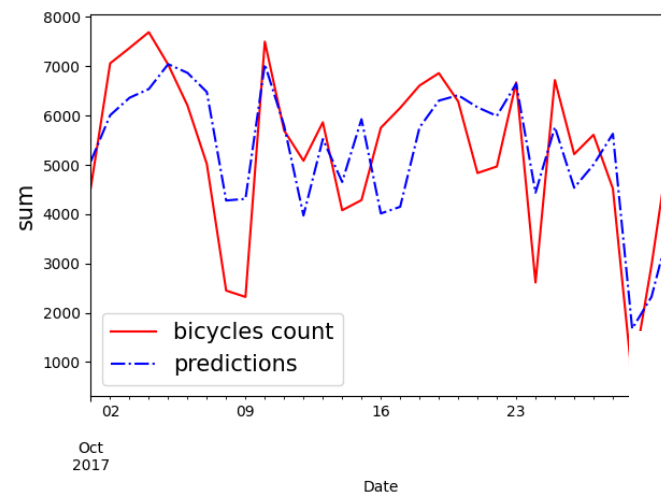
Evaluation of Forecasts

NY bridges bicycles count

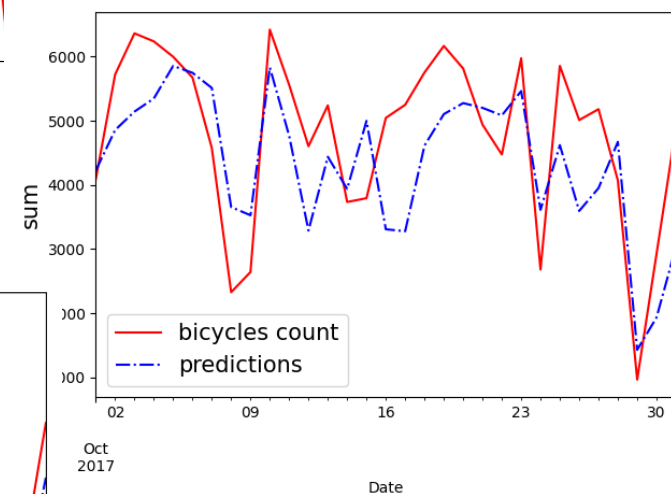
total



Manhattan bridge

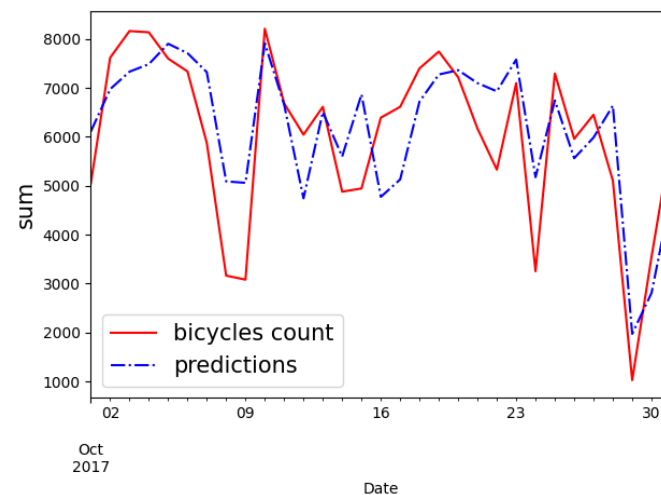


Brooklyn bridge



Queensboro bridge

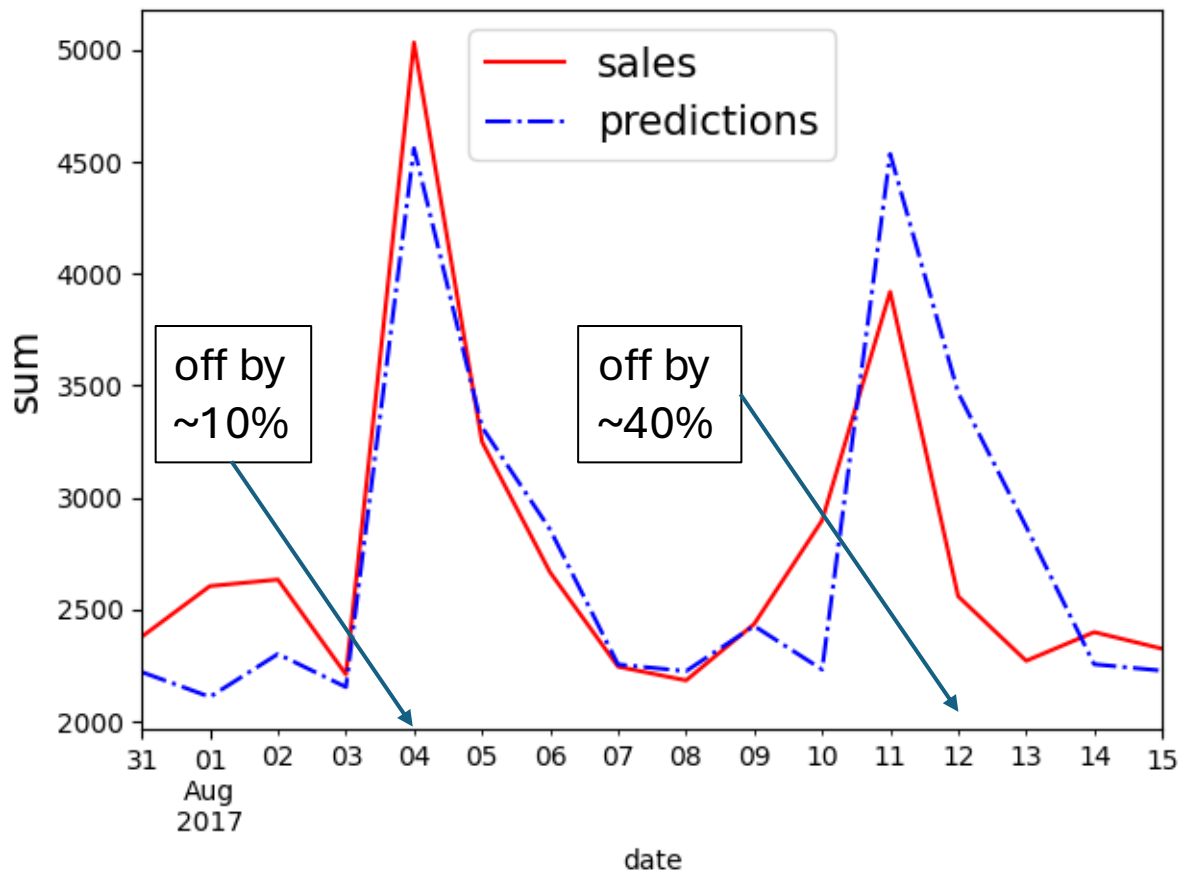
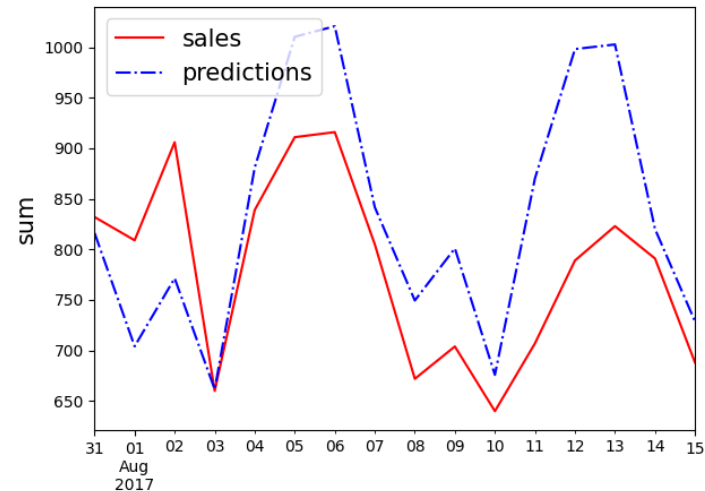
Williamsburg bridge



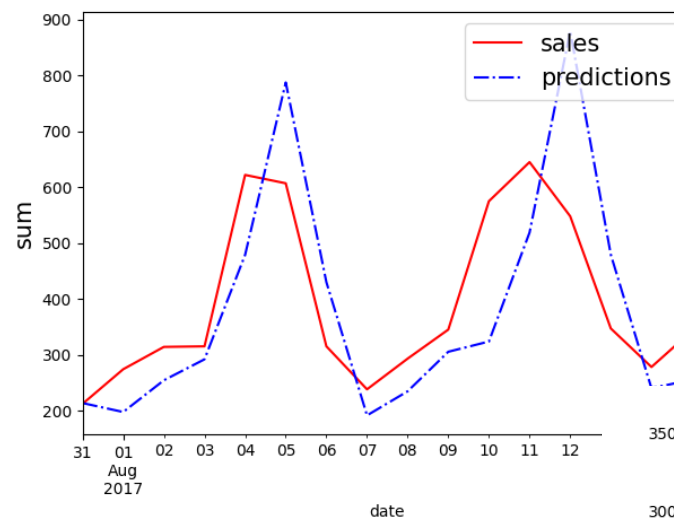
Store sales

total

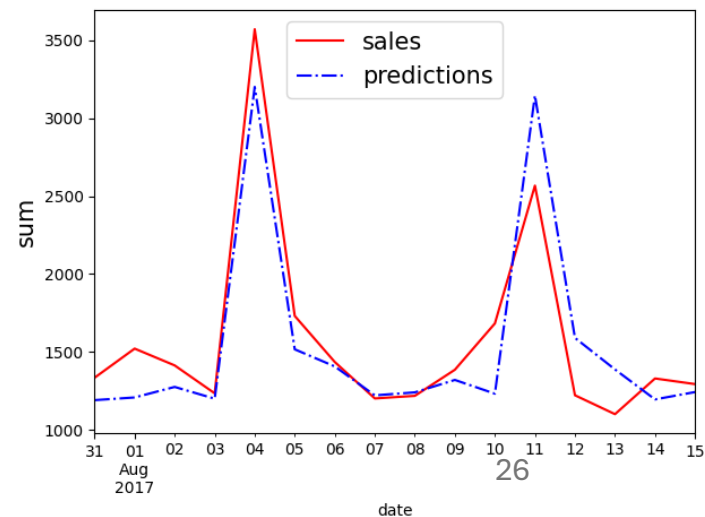
sum over several stores



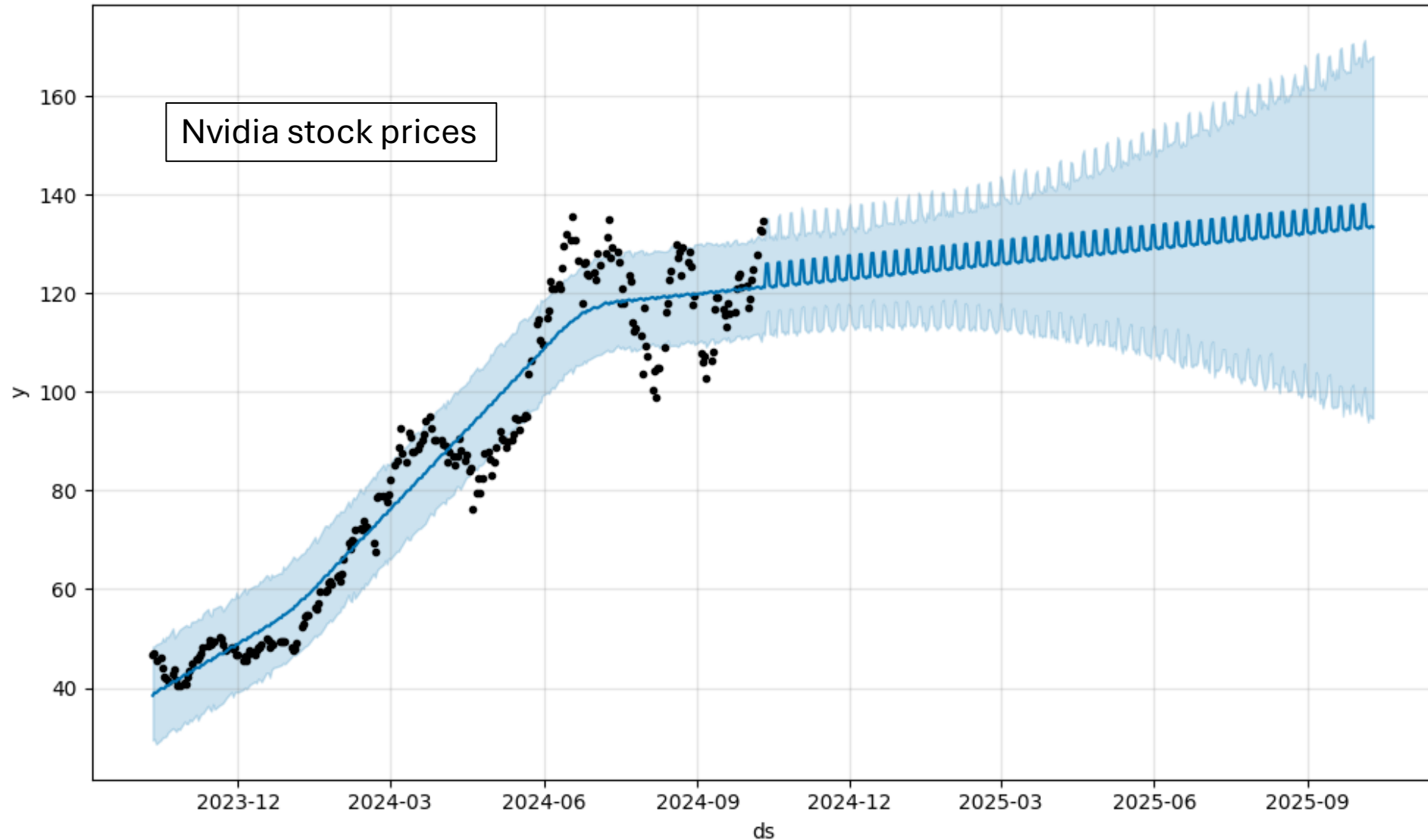
liquor, wine, beer



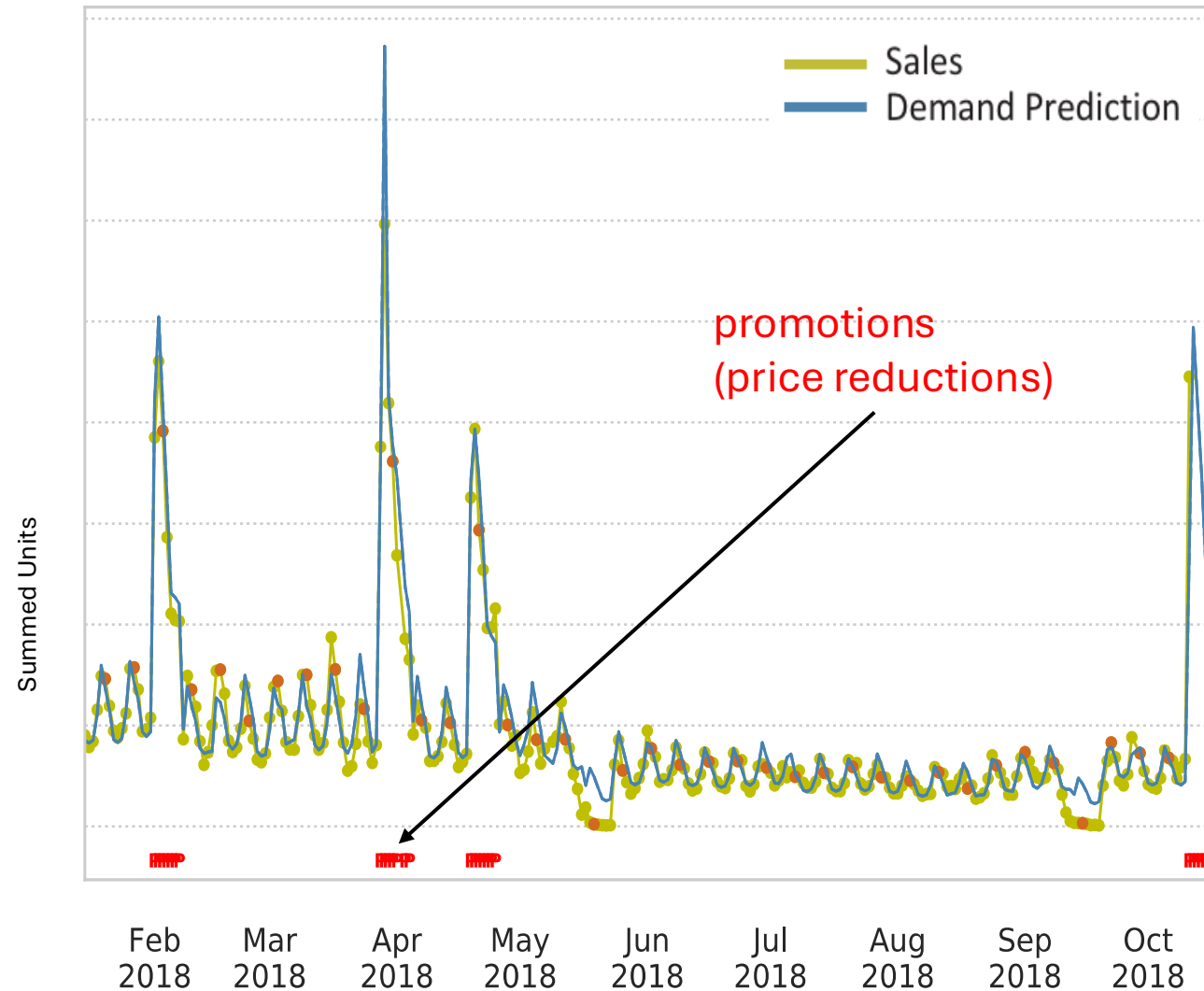
eggs



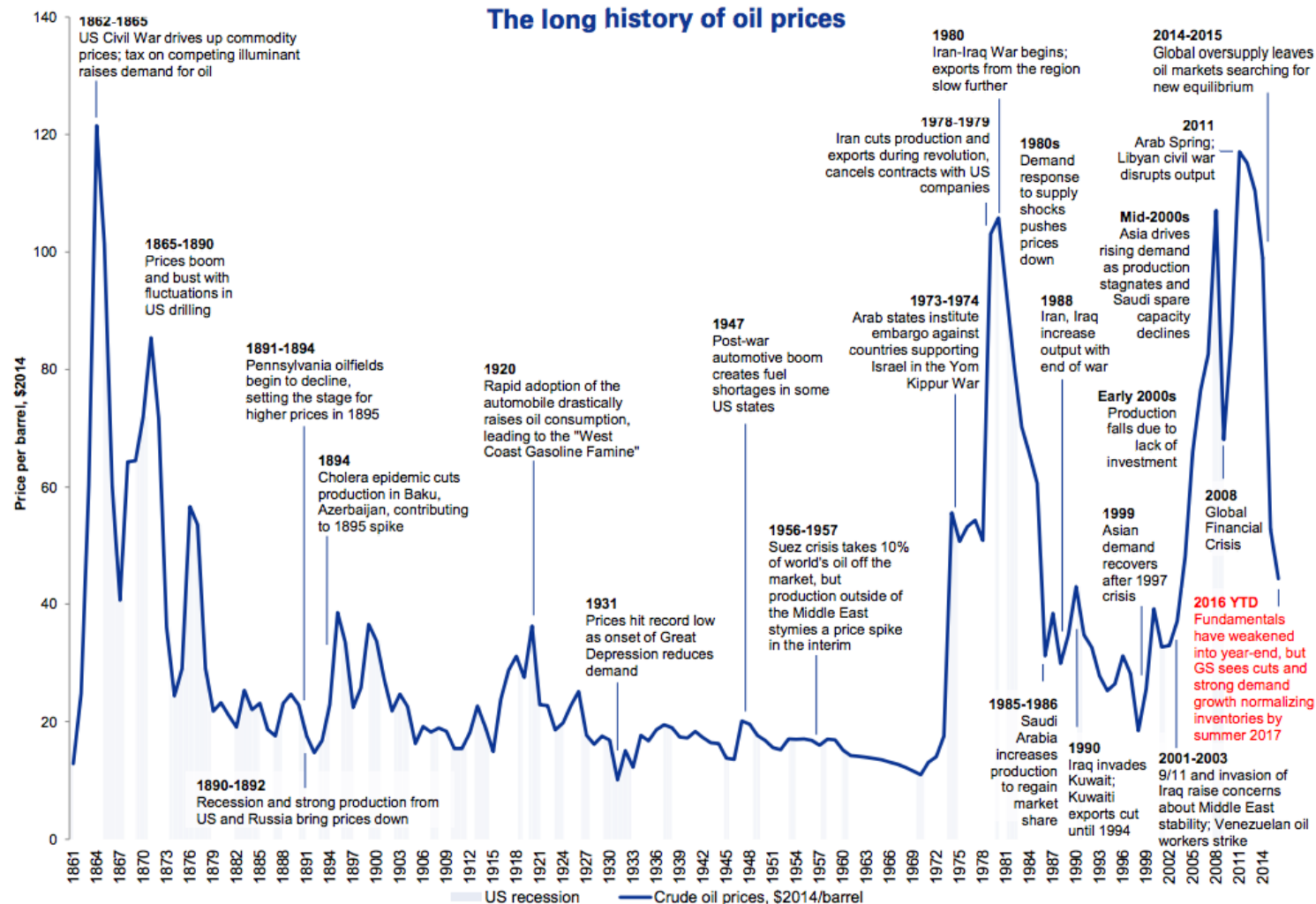
Uncertainty increases with forecast horizon



You can predict effects of foreseeable events



But you cannot predict effects of unforeseeable events



An earlier version of this chart appeared on pg. 16 of *Top of Mind Issue #52: OPEC and Oil Opportunities*.

Note: 2016 price shown is YTD average as of Dec. 19, 2016.

Source for data: BP, NBER/Federal Reserve Bank of St. Louis, Haver Analytics.

Source for annotations: ©James Hamilton, "Historical Oil Shocks," University of California, San Diego, February 2011; various news sources; Goldman Sachs Global Investment Research.

Deep Learning Methods for Sequential Data

→ direct inclusion of auto-correlation in ML model (but beware: mainly spurious)

convolutional neural networks (CNN)

- main application computer vision: images correspond to regular 2D grids of data (pixels)
- time series data as 1D grid at regular time intervals
- parameter sharing via convolutions (and pooling) → highly regularized feed-forward networks
- fixed time window (kernel)
- multivariate time series model via different input channels

recurrent neural networks (RNN)

- main application natural language processing (NLP): sequence of words (tokens)
- time series data as time-tagged stream
- parameter sharing via recurrent nodes → context-awareness (input from past activations)
- variable-length time series
- multivariate time series model via more input nodes

transformers

- main application NLP (largely replaced RNNs), but also computer vision
- time series data as directed graph (attention in edges)
- masked self-attention to focus on relevant past time steps and positional encoding to include order of sequence
- multivariate time series model via input concatenation

Anomaly Detection